

SIMATS ENGINEERING
ASSESSMENT TOOL 1

R Harika (192572140)

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool: Code Challenge (High)

Assessment Tool Weightage: 35%

QUESTIONS

Total Marks: 50

Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 –Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 –Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 –Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 –Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 –Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 –Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 –Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 –Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 –Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 –Analyze	5

Code of Conduct

I _Rasineni Harika(192572140)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Harika

Rubrics for Calculation

Criteria	Weight age	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/ inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
TestCase Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

ANSWERS

Q1. Heap File Organization (Insert, Delete, Sequential Scan)

```
class HeapFile:
    def __init__(self):
        self.records = []
    def insert(self, record):
        self.records.append(record)
        print(f"Inserted: {record}")
    def delete(self, key):
        for r in self.records:
            if r['id'] == key:
                self.records.remove(r)
                print(f"Deleted record with id {key}")
                return
        print(f"Record with id {key} not found")
    def sequential_scan(self):
        print("Sequential Scan Result:")
        for r in self.records:
            print(r)
if __name__ == "__main__":
    hf = HeapFile()
    hf.insert({'id': 1, 'name': 'Alice'})
    hf.insert({'id': 2, 'name': 'Bob'})
    hf.insert({'id': 3, 'name': 'Charlie'})
    hf.sequential_scan()
    hf.delete(2)
    hf.sequential_scan()
```

Explanation: A heap file stores records in insertion order with no particular sequencing. Insert appends to the list ($O(1)$), delete requires a linear scan ($O(n)$), and sequential scan simply iterates through all records ($O(n)$).

Q2. Static Hashing with Chaining (500 Records)

```
class StaticHashTable:
    def __init__(self, size=500):
        self.size = size
        self.table = [[] for _ in range(size)]
    def _hash(self, key):
        return key % self.size
    def insert(self, key, value):
        idx = self._hash(key)
        self.table[idx].append((key, value))
    def search(self, key):
        idx = self._hash(key)
        for k, v in self.table[idx]:
            if k == key:
```

```

        return v
    return None
def display_bucket(self, key):
    idx = self._hash(key)
    print(f"Bucket {idx}: {self.table[idx]}")
if __name__ == "__main__":
    ht = StaticHashTable(500)
    students = [(101, "Alice"), (601, "Bob"), (1101, "Charlie")]
    for sid, name in students:
        ht.insert(sid, name)
    print("Search 101:", ht.search(101))
    print("Search 601:", ht.search(601))
    ht.display_bucket(101)

```

Explanation: Static hashing uses a fixed number of buckets (500). The hash function $\text{key} \% \text{size}$ maps a key to a bucket index. Collisions are resolved using chaining — each bucket is a list, and colliding entries are appended to that list.

Q3. Sorted File Organization with Binary Search

```

class SortedFile:
    def __init__(self):
        self.records = []
    def insert(self, record):
        self.records.append(record)
        self.records.sort(key=lambda r: r['id'])
    def binary_search(self, key):
        low, high = 0, len(self.records) - 1
        while low <= high:
            mid = (low + high) // 2
            if self.records[mid]['id'] == key:
                return self.records[mid]
            elif self.records[mid]['id'] < key:
                low = mid + 1
            else:
                high = mid - 1
        return None
if __name__ == "__main__":
    sf = SortedFile()
    for rec in [{ 'id': 30, 'name': 'C' }, { 'id': 10, 'name': 'A' }, { 'id': 20, 'name': 'B' }]:
        sf.insert(rec)
    print("Sorted records:", sf.records)
    print("Search id=20:", sf.binary_search(20))
    print("Search id=99:", sf.binary_search(99))

```

Explanation: Records are kept sorted by primary key at all times. Because the file is sorted, retrieval by key uses binary search — $O(\log n)$ — instead of a linear scan, at the cost of more expensive inserts ($O(n)$ to maintain order).

Q4. B-Tree of Order 4 (Insert, Search, Display)

```

class BTreeNode:
    def __init__(self, leaf=True):
        self.keys = []
        self.children = []
        self.leaf = leaf

class BTree:
    def __init__(self, order=4):
        self.root = BTreeNode(True)
        self.order = order

    def search(self, key, node=None):
        node = node or self.root
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        if i < len(node.keys) and node.keys[i] == key:
            return True
        if node.leaf:
            return False
        return self.search(key, node.children[i])

    def insert(self, key):
        root = self.root
        if len(root.keys) == self.order - 1:
            new_root = BTreeNode(False)
            new_root.children.append(root)
            self._split_child(new_root, 0)
            self.root = new_root
        self._insert_non_full(self.root, key)

    def _insert_non_full(self, node, key):
        i = len(node.keys) - 1
        if node.leaf:
            node.keys.append(None)
            while i >= 0 and key < node.keys[i]:
                node.keys[i + 1] = node.keys[i]
                i -= 1
            node.keys[i + 1] = key
        else:
            while i >= 0 and key < node.keys[i]:
                i -= 1
            i += 1
            if len(node.children[i].keys) == self.order - 1:
                self._split_child(node, i)
                if key > node.keys[i]:
                    i += 1
            self._insert_non_full(node.children[i], key)

    def _split_child(self, parent, i):
        order = self.order
        child = parent.children[i]
        mid = (order - 1) // 2

```

```

new_node = BTreeNode(child.leaf)
new_node.keys = child.keys[mid + 1:]
up_key = child.keys[mid]
child.keys = child.keys[:mid]
if not child.leaf:
    new_node.children = child.children[mid + 1:]
    child.children = child.children[:mid + 1]
parent.keys.insert(i, up_key)
parent.children.insert(i + 1, new_node)
def display(self, node=None, level=0):
    node = node or self.root
    print(" " * level + str(node.keys))
    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)
if __name__ == "__main__":
    bt = BTree(order=4)
    for k in [10, 20, 5, 6, 12, 30, 7, 17]:
        bt.insert(k)
    print("B-Tree structure:")
    bt.display()
    print("Search 12:", bt.search(12))
    print("Search 99:", bt.search(99))

```

Explanation: A B-Tree of order 4 allows a maximum of 3 keys and 4 children per node. Insertion always happens at a leaf; if a node overflows it splits, pushing the middle key up to the parent — keeping the tree balanced.

Q5. B+ Tree with Leaf-Level Linking (Range Queries)

```

class BPlusNode:
    def __init__(self, leaf=True):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.values = []
        self.next = None
class BPlusTree:
    def __init__(self, order=4):
        self.root = BPlusNode(True)
        self.order = order
    def insert(self, key, value):
        root = self.root
        if len(root.keys) == self.order - 1:
            new_root = BPlusNode(False)
            new_root.children.append(root)
            self._split(new_root, 0)
            self.root = new_root
        self._insert_non_full(self.root, key, value)

```

```

def _insert_non_full(self, node, key, value):
    if node.leaf:
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        node.keys.insert(i, key)
        node.values.insert(i, value)
    else:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1
        if len(node.children[i].keys) == self.order - 1:
            self._split(node, i)
            if key >= node.keys[i]:
                i += 1
            self._insert_non_full(node.children[i], key, value)
def _split(self, parent, i):
    child = parent.children[i]
    mid = len(child.keys) // 2
    new_node = BPlusNode(child.leaf)
    if child.leaf:
        new_node.keys = child.keys[mid:]
        new_node.values = child.values[mid:]
        child.keys = child.keys[:mid]
        child.values = child.values[:mid]
        new_node.next = child.next
        child.next = new_node
        up_key = new_node.keys[0]
    else:
        up_key = child.keys[mid]
        new_node.keys = child.keys[mid + 1:]
        new_node.children = child.children[mid + 1:]
        child.keys = child.keys[:mid]
        child.children = child.children[:mid + 1]
    parent.keys.insert(i, up_key)
    parent.children.insert(i + 1, new_node)
def range_query(self, low, high):
    node = self.root
    while not node.leaf:
        i = 0
        while i < len(node.keys) and low >= node.keys[i]:
            i += 1
        node = node.children[i]
    result = []
    while node:
        for k, v in zip(node.keys, node.values):
            if low <= k <= high:
                result.append((k, v))

```

```

        node = node.next
    return result
if __name__ == "__main__":
    bpt = BPlusTree(order=4)
    for k in [10, 20, 5, 6, 12, 30, 7, 17]:
        bpt.insert(k, f'val{k}')
    print("Range query 6 to 20:", bpt.range_query(6, 20))

```

Explanation: All actual records live in leaf nodes, and every leaf points to the next leaf, forming a linked list. This makes range queries efficient: find the starting leaf via one root-to-leaf traversal, then scan forward through linked leaves.

Q6. Dense Primary Index on a Sorted File

```

class DenseIndex:
    def __init__(self):
        self.data_file = []
        self.index = {}
    def build(self, records):
        self.data_file = sorted(records, key=lambda r: r['id'])
        self.index = {r['id']: pos for pos, r in enumerate(self.data_file)}
    def search(self, key):
        pos = self.index.get(key)
        if pos is not None:
            return self.data_file[pos]
        return None
    def display_index(self):
        print("Index Table (Key -> Block Position):")
        for k, v in self.index.items():
            print(f'{k} -> {v}')
if __name__ == "__main__":
    di = DenseIndex()
    di.build([
        {'id': 30, 'name': 'C'}, {'id': 10, 'name': 'A'},
        {'id': 20, 'name': 'B'}, {'id': 40, 'name': 'D'}
    ])
    di.display_index()
    print("Search id=20:", di.search(20))

```

Explanation: A dense index has one index entry for every record in the data file, each pointing to that record's exact position/block. Search is a direct lookup instead of scanning the data file.

Q7. Extendible Hashing with Directory Doubling

```

class ExtendibleHashing:
    def __init__(self, bucket_size=2):
        self.global_depth = 1
        self.bucket_size = bucket_size
        self.directory = {'0': [], '1': []}
        self.local_depth = {'0': 1, '1': 1}

```



```

def _hash(self, key):
    return bin(key)[2:].zfill(self.global_depth)[-self.global_depth:]
def insert(self, key):
    idx = self._hash(key)
    bucket = self.directory[idx]
    if len(bucket) < self.bucket_size:
        bucket.append(key)
        print(f'Inserted {key} into bucket {idx}')
        return
    local_d = self.local_depth[idx]
    if local_d == self.global_depth:
        print("Overflow -> Doubling directory")
        new_dir = {}
        for k, v in self.directory.items():
            new_dir['0' + k] = v
            new_dir['1' + k] = v
        self.directory = new_dir
        self.global_depth += 1
    self.local_depth[idx] += 1
    new_local_d = self.local_depth[idx]
    old_keys = self.directory[idx][:]
    b0, b1 = [], []
    for k in old_keys + [key]:
        suffix = bin(k)[2:].zfill(new_local_d)[-new_local_d:]
        if suffix.endswith('0'):
            b0.append(k)
        else:
            b1.append(k)
    for dkey in list(self.directory.keys()):
        if dkey[-local_d:] == idx[-local_d:]:
            if dkey[-1] == '0':
                self.directory[dkey] = b0
            else:
                self.directory[dkey] = b1
        self.local_depth[dkey] = new_local_d
    print(f'Split bucket {idx} -> {b0} | {b1}')
def display(self):
    print("Directory:")
    for k, v in self.directory.items():
        print(f'{k} -> {v}')
if __name__ == "__main__":
    eh = ExtendibleHashing(bucket_size=2)
    for k in [4, 12, 8, 5, 16, 1, 9, 20]:
        eh.insert(k)
    eh.display()

```

Explanation: Extendible hashing keeps a directory of pointers to buckets, indexed by the last `global_depth` bits of the hash. When a bucket overflows, if its local depth equals the global

depth, the entire directory doubles so a new bit can distinguish the split buckets.

Q8. Secondary Storage Block Access — I/O Cost Comparison

```
class StorageIOSimulator:
    def __init__(self, num_records, block_factor):
        self.num_records = num_records
        self.block_factor = block_factor
    def sequential_scan_io(self):
        return -(self.num_records // self.block_factor) # ceil division
    def indexed_search_io(self, index_height=2):
        return index_height + 1
if __name__ == "__main__":
    sim = StorageIOSimulator(num_records=10000, block_factor=50)
    seq_io = sim.sequential_scan_io()
    idx_io = sim.indexed_search_io(index_height=3)
    print(f"Sequential Scan I/O operations: {seq_io}")
    print(f"Indexed Retrieval I/O operations: {idx_io}")
    print(f"Indexed retrieval is {seq_io / idx_io:.1f}x faster in block accesses")
```

Explanation: Sequential retrieval must read every block that stores the file. Indexed retrieval only reads the blocks along the index tree's height, plus one block for the record itself — far fewer I/Os for large files.

Q9. Sparse Index vs. Dense Index — Timing Comparison

```
import time
import random
class SparseIndex:
    def __init__(self, block_size=10):
        self.data_file = []
        self.index = []
        self.block_size = block_size
    def build(self, records):
        self.data_file = sorted(records, key=lambda r: r['id'])
        for i in range(0, len(self.data_file), self.block_size):
            self.index.append((self.data_file[i]['id'], i))
    def search(self, key):
        lo, hi = 0, len(self.index) - 1
        block_pos = 0
        while lo <= hi:
            mid = (lo + hi) // 2
            if self.index[mid][0] <= key:
                block_pos = self.index[mid][1]
                lo = mid + 1
            else:
                hi = mid - 1
        for r in self.data_file[block_pos: block_pos + self.block_size]:
            if r['id'] == key:
                return r
```

```

    return None
class DenseIndex:
    def __init__(self):
        self.data_file = []
        self.index = {}
    def build(self, records):
        self.data_file = sorted(records, key=lambda r: r['id'])
        self.index = {r['id']: pos for pos, r in enumerate(self.data_file)}
    def search(self, key):
        pos = self.index.get(key)
        return self.data_file[pos] if pos is not None else None
if __name__ == "__main__":
    records = [{'id': i, 'name': f'rec{i}'} for i in range(100000)]
    random.shuffle(records)
    sparse = SparseIndex(block_size=20)
    sparse.build(records)
    dense = DenseIndex()
    dense.build(records)
    target = 54321
    t0 = time.perf_counter()
    sparse.search(target)
    t1 = time.perf_counter()
    dense.search(target)
    t2 = time.perf_counter()
    print(f"Sparse index search time: {(t1 - t0) * 1e6:.2f} microseconds")
    print(f"Dense index search time: {(t2 - t1) * 1e6:.2f} microseconds")

```

Explanation: A sparse index stores one entry per block, so it's smaller and fits in memory more easily, but a match requires a binary search on the index plus a scan within the block. A dense index gives faster direct lookups at the cost of more index storage.

Q10. Construct&Traverse B+ Tree, Range Query (15–45)

```

class BPlusNode:
    def __init__(self, leaf=True):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.values = []
        self.next = None
class BPlusTree:
    def __init__(self, order=4):
        self.root = BPlusNode(True)
        self.order = order
    def insert(self, key, value=None):
        value = value if value is not None else f'rec{key}'
        root = self.root
        if len(root.keys) == self.order - 1:
            new_root = BPlusNode(False)

```

```

        new_root.children.append(root)
        self._split(new_root, 0)
        self.root = new_root
    self._insert_non_full(self.root, key, value)
def _insert_non_full(self, node, key, value):
    if node.leaf:
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        node.keys.insert(i, key)
        node.values.insert(i, value)
    else:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1
        if len(node.children[i].keys) == self.order - 1:
            self._split(node, i)
            if key >= node.keys[i]:
                i += 1
        self._insert_non_full(node.children[i], key, value)
def _split(self, parent, i):
    child = parent.children[i]
    mid = len(child.keys) // 2
    new_node = BPlusNode(child.leaf)
    if child.leaf:
        new_node.keys = child.keys[mid:]
        new_node.values = child.values[mid:]
        child.keys = child.keys[:mid]
        child.values = child.values[:mid]
        new_node.next = child.next
        child.next = new_node
        up_key = new_node.keys[0]
    else:
        up_key = child.keys[mid]
        new_node.keys = child.keys[mid + 1:]
        new_node.children = child.children[mid + 1:]
        child.keys = child.keys[:mid]
        child.children = child.children[:mid + 1]
    parent.keys.insert(i, up_key)
    parent.children.insert(i + 1, new_node)
def traverse(self, node=None, level=0):
    node = node or self.root
    print(" " * level + f"Level {level}: {node.keys}")
    if not node.leaf:
        for child in node.children:
            self.traverse(child, level + 1)
def range_query(self, low, high):
    node = self.root

```

```

while not node.leaf:
    i = 0
    while i < len(node.keys) and low >= node.keys[i]:
        i += 1
    node = node.children[i]
result = []
while node:
    for k, v in zip(node.keys, node.values):
        if low <= k <= high:
            result.append((k, v))
    node = node.next
return result
if __name__ == "__main__":
    keys = [5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
    bpt = BPlusTree(order=4)
    for k in keys:
        bpt.insert(k)
    print("B+ Tree Traversal:")
    bpt.traverse()
    print("Range Query [15, 45]:")
    for k, v in bpt.range_query(15, 45):
        print(f'{k}: {v}')

```

Explanation: The tree is built the same way as Q5. range_query descends once to the leaf containing the lower bound, then follows the next pointers across leaves, collecting all keys within [15, 45].